

Compiladores e Intérpretes

Pautas Etapa 5

Generación de Código

Universidad Nacional del Sur
Departamento de Ciencias e Ingeniería de la Computación
2025

Introducción - Aclaraciones

- El desarrollo de esta **etapa** del proyecto se **corresponde** con los **Tema 5.2, 5.3.1, 5.3.2, 5.3.3 y 5.4** de la materia
- Te recomendamos que **hayas mirado y tengas frescos** todos los temas allí tratados **antes de arrancar** el diseño/desarrollo de esta etapa
- También te **esencial** mirar el documento de **Apunte CelVM y el zip con la Maquina Virtual**
- En los ejemplos de los diseños de esta presentación vas a ver que hay "...", eso quiere decir que es muy posible que a el diseño tenga algunas/varias cosas mas

Introducción

- En esta **etapa** hay que **diseñar e implementar** un **el Generador de Código** para la CelVM del compilador de **MiniJava**
- Para esto abra que extender las funcionalidades de la **TS** y los **nodos del AST** para que vayan generando el código de lo que están representando
- Además, de manera similar a la etapa anterior va ser necesario un **Modulo Principal** que:
 - Manejará la **interfaz** con el **usuario** y
 - **Reportará los errores** detectados por el Semántico, el Sintáctico y el Léxico
 - **Generará** el archivo de salida con el **código CeiVM generado**
 - En caso de no haber errores indicará que el análisis fue exitoso

Diseño de la Generación de Código

Diseño de la Generación de Código

- Como pueden ver en el Tema 5.4 la generación de código se hace **recorriendo** la **TS** y los **ASTs**
 - La mayoría de la generación de código lineal se **diseña** como un **recorrido del AST**
 - A cada **nodo** se le asocia la responsabilidad de **generar el código** correspondiente a **su significado**
 - La **generación de código** en un nodo **depende** de que se hayan **resuelto los nombres** (id) utilizados en el nodo

Diseño de la Generación de Código

- Para saber que código generar en cada nodo es importante:
 - **Entender** como funciona la CelVM
 - **Entender** como se **traducen accesos a variables, llamadas, etc**

Para eso te recomendamos mirar los temas **5.3.1, 5.3.2 y 5.3.3** y el apunte de la **CelVM**

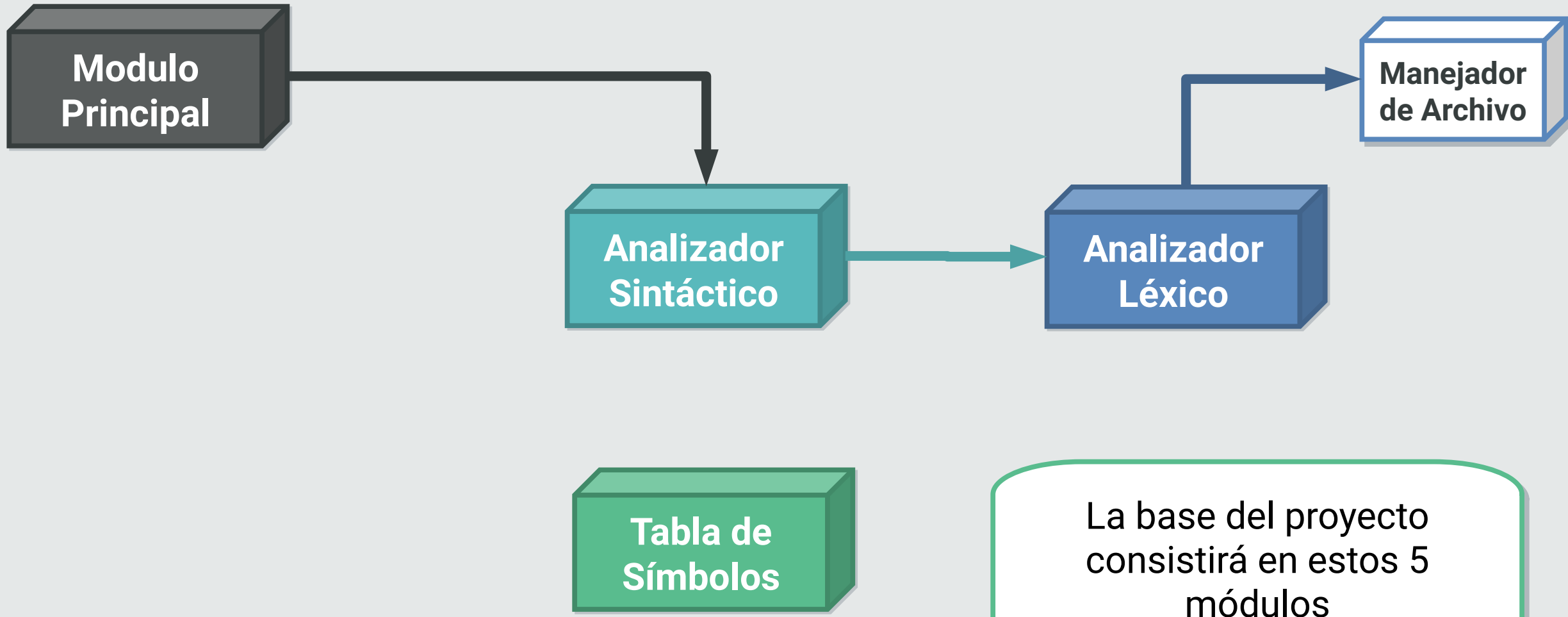
Herramientas de Test para la Evaluación

- En la tarea también les adjuntamos **herramientas de testing** de JUnit ***DISTINTA A LAS ETAPAS ANTERIORES!!!!***
- Estas herramientas las utilizamos en la cátedra como parte de la **evaluación** de los proyectos
- En el adjunto van a encontrar una **guía** de como agregarlas al proyecto (está armada respecto a **IntelliJ IDEA**)

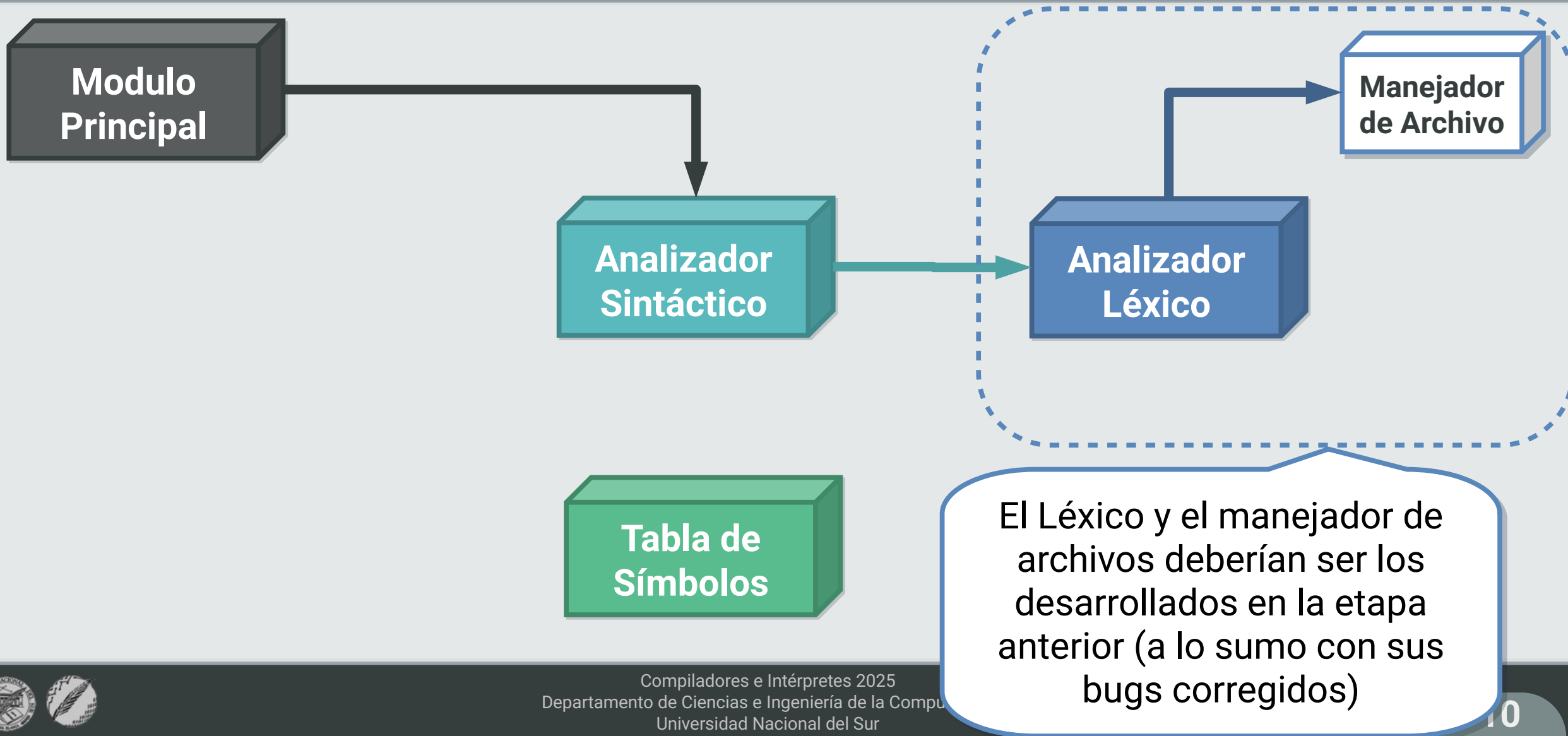
Aclaración Importante: Las herramientas **no incluyen** los **casos de prueba** necesarios para **verificar adecuadamente** el compilador. El desarrollo de los casos es **responsabilidad del estudiante**. *Ademas, estas no son las únicas herramientas que utiliza la cátedra para la evaluación*

Estructura y Funcionalidad del Software a Desarrollar

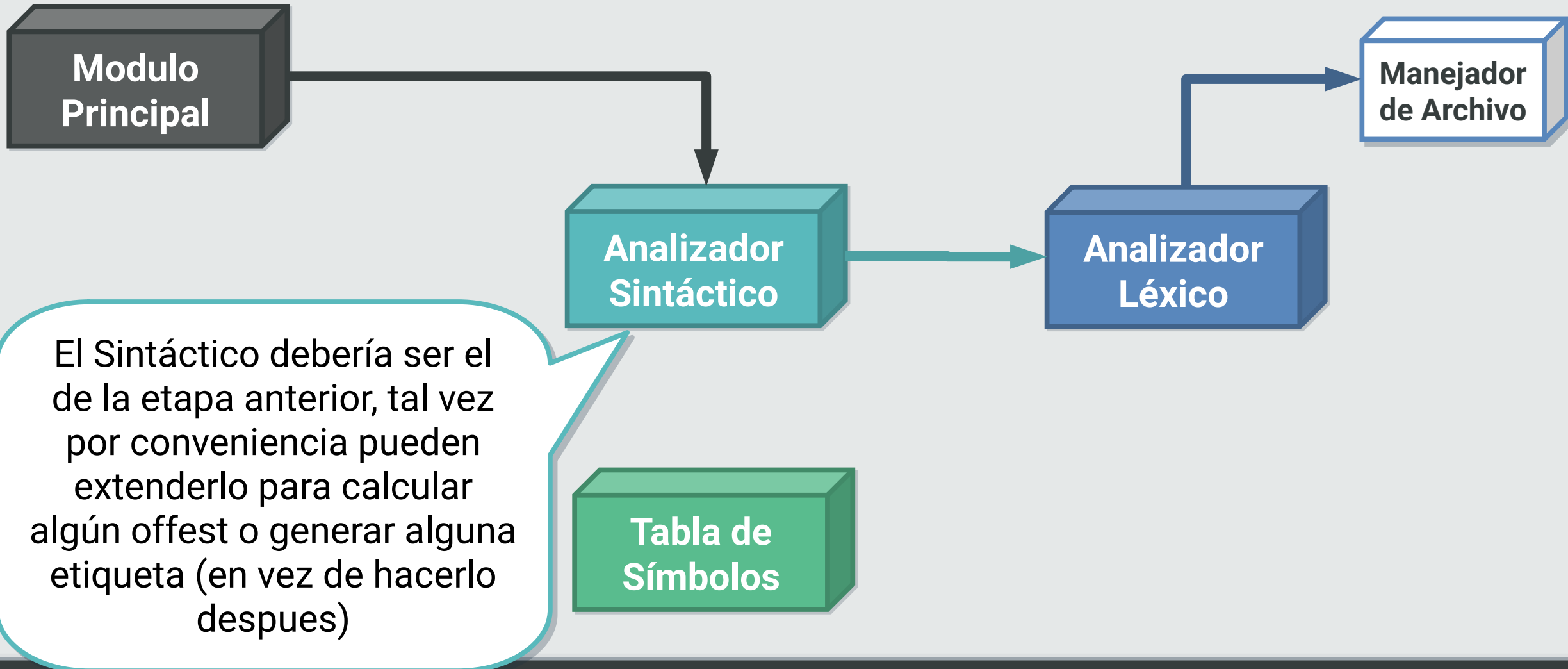
Esquema General



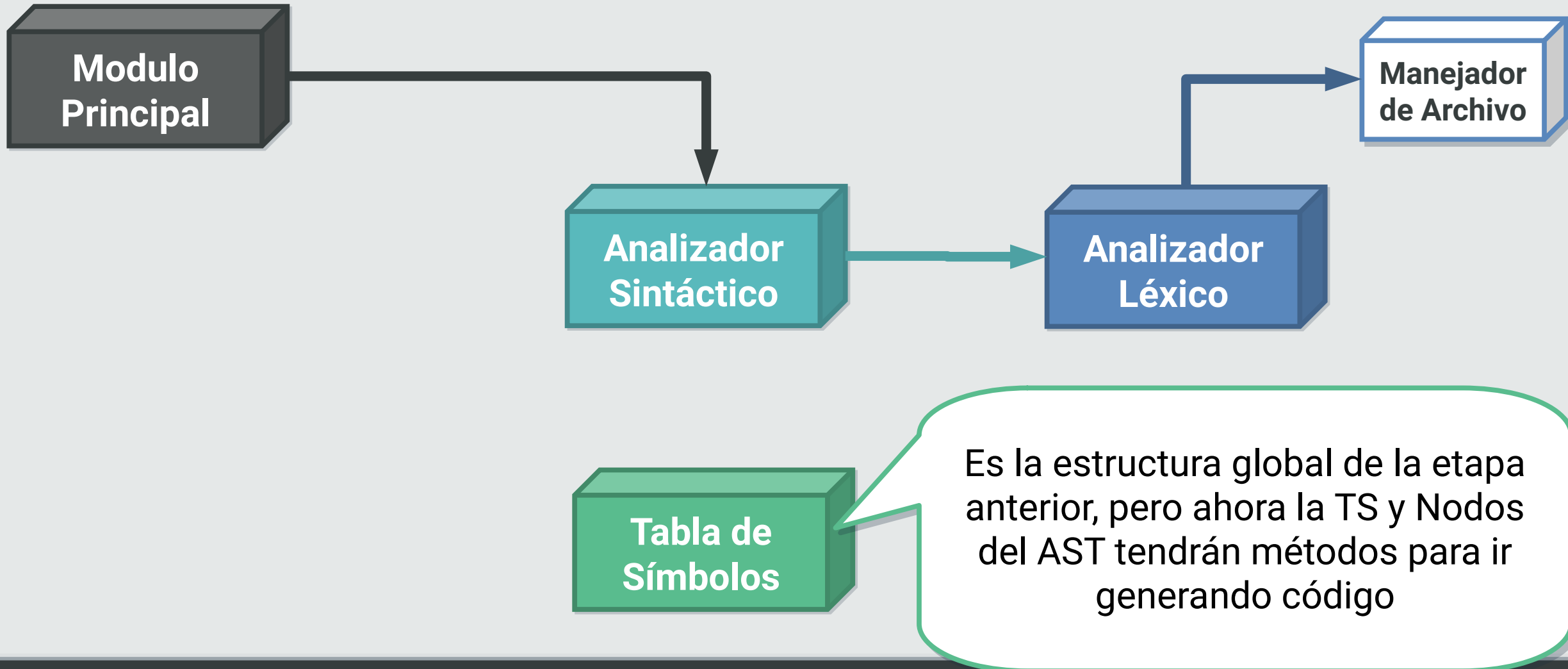
Esquema General



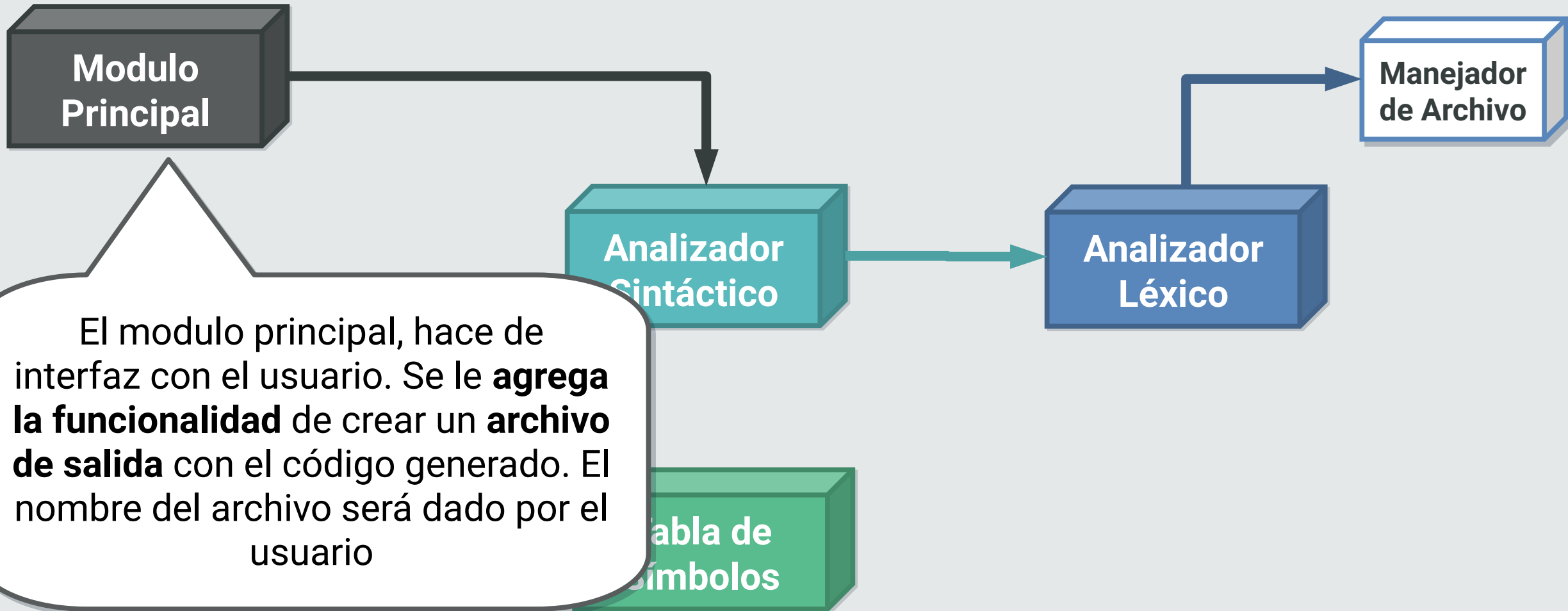
Esquema General



Esquema General



Esquema General



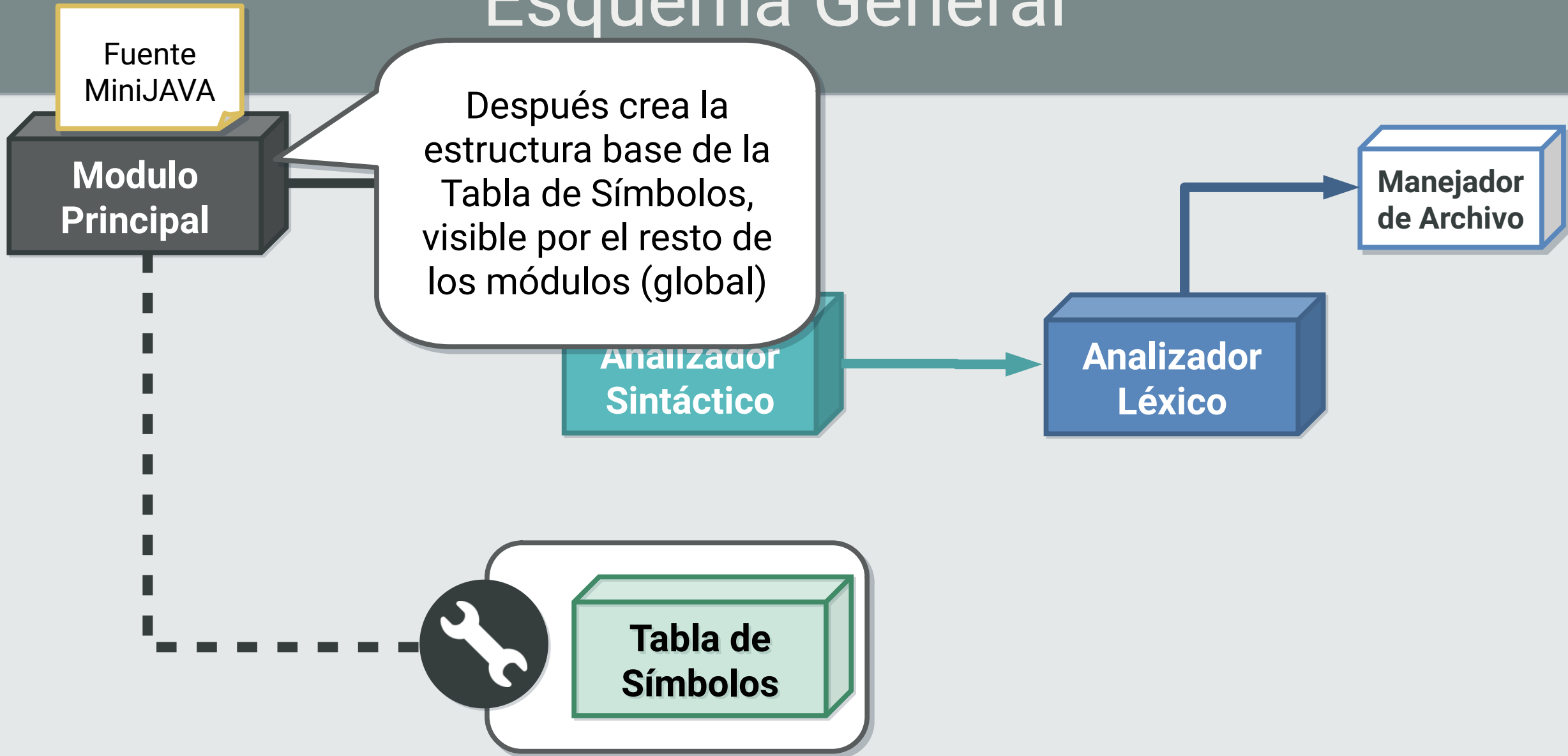
Esquema General

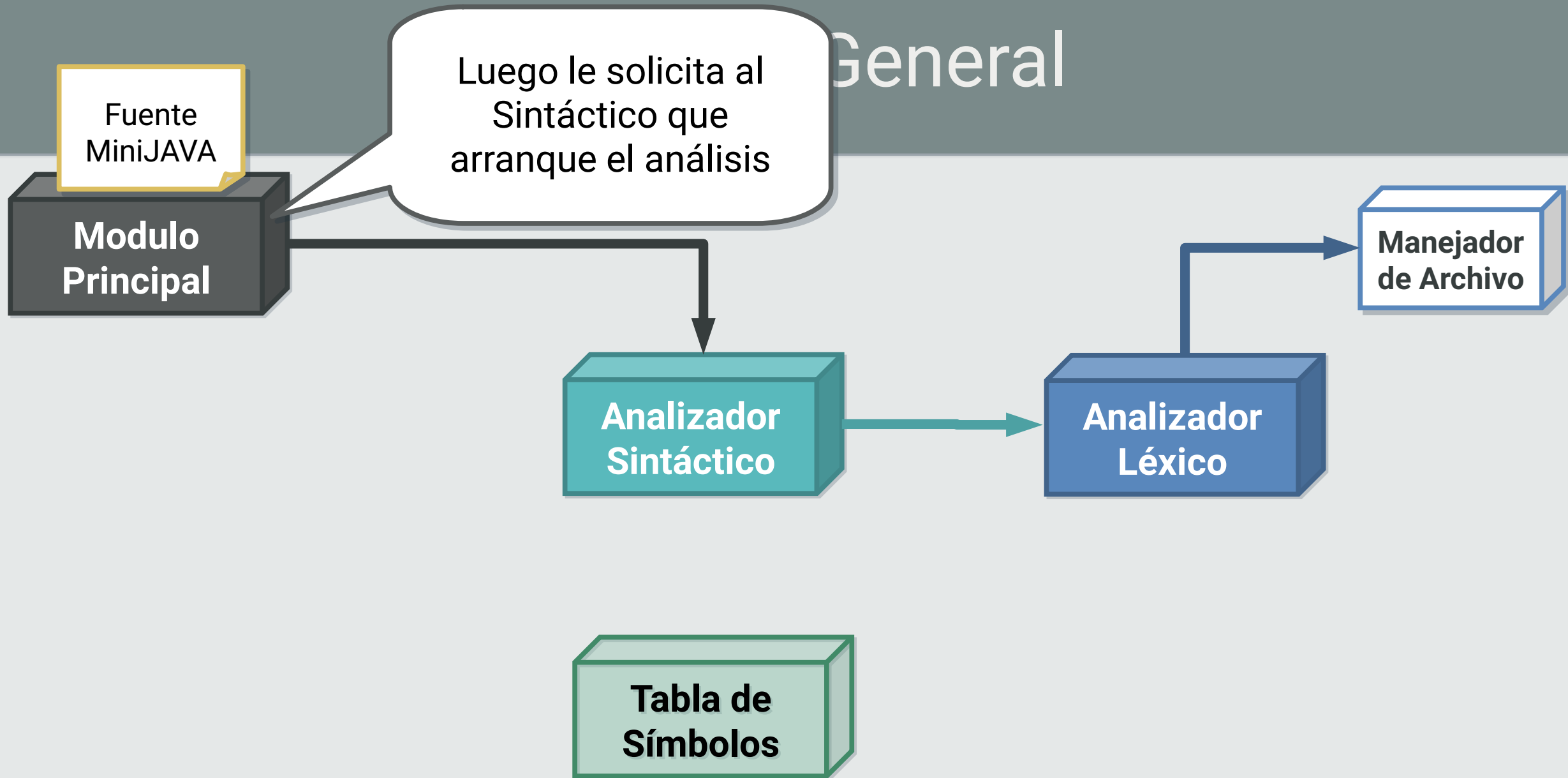


Esquema General

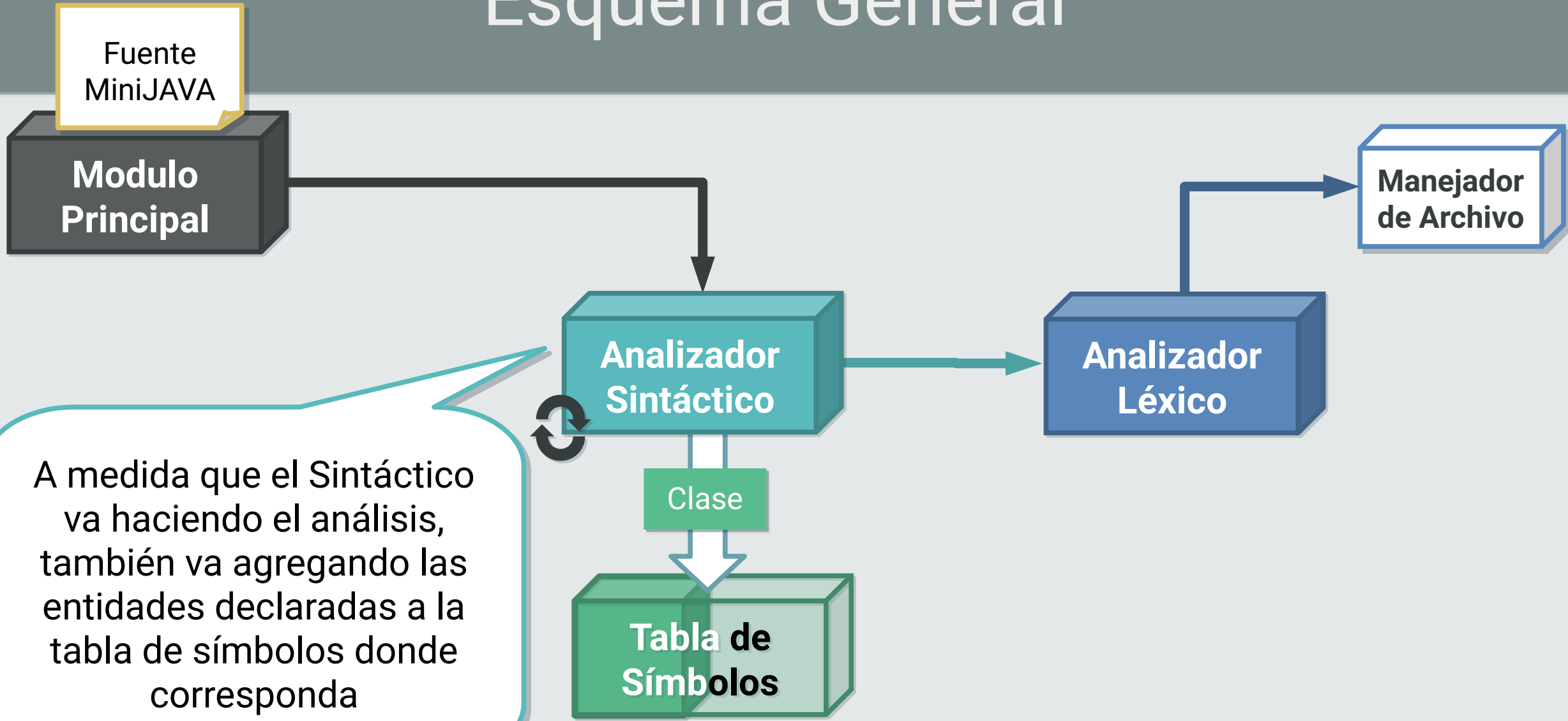


Esquema General

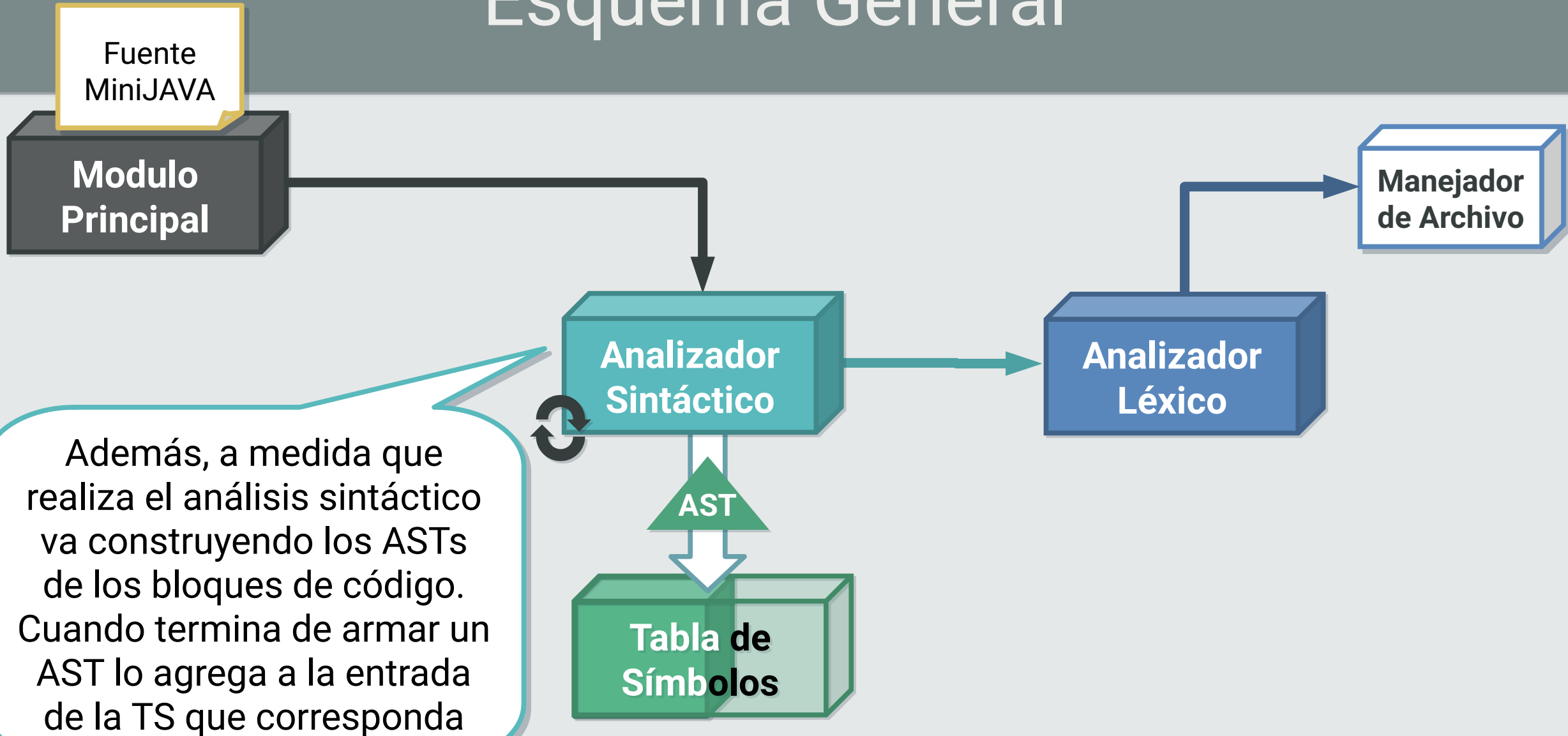


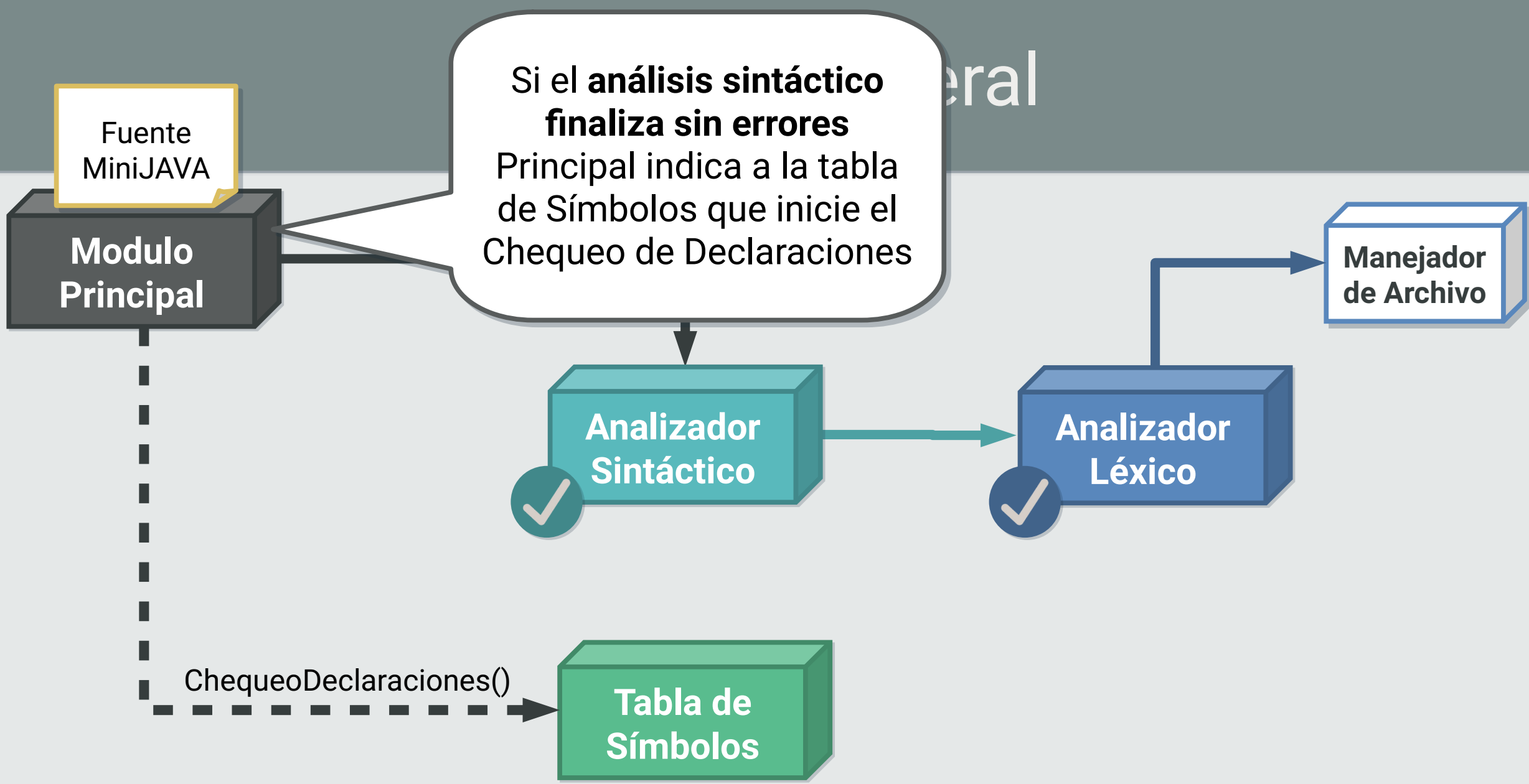


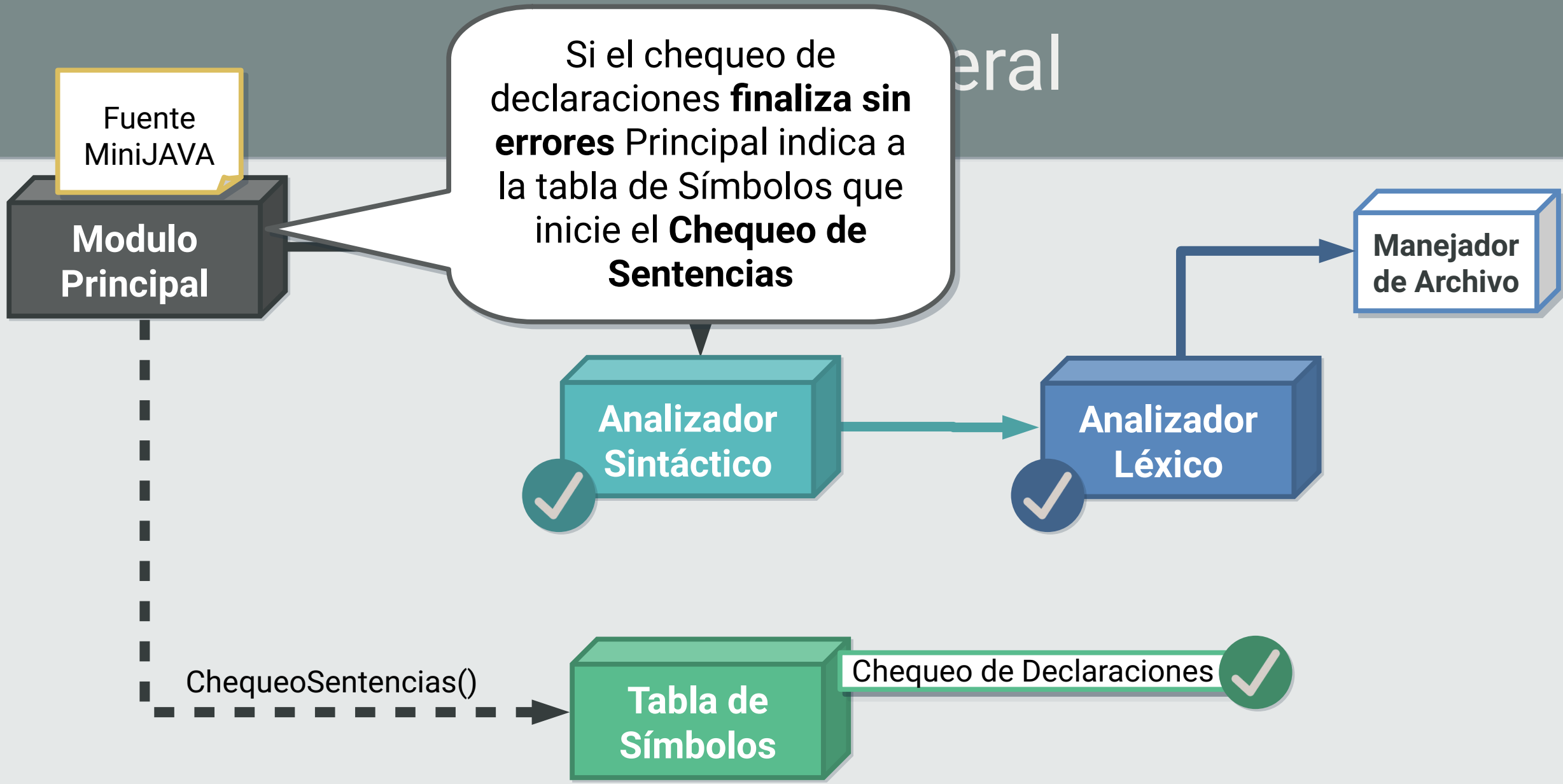
Esquema General

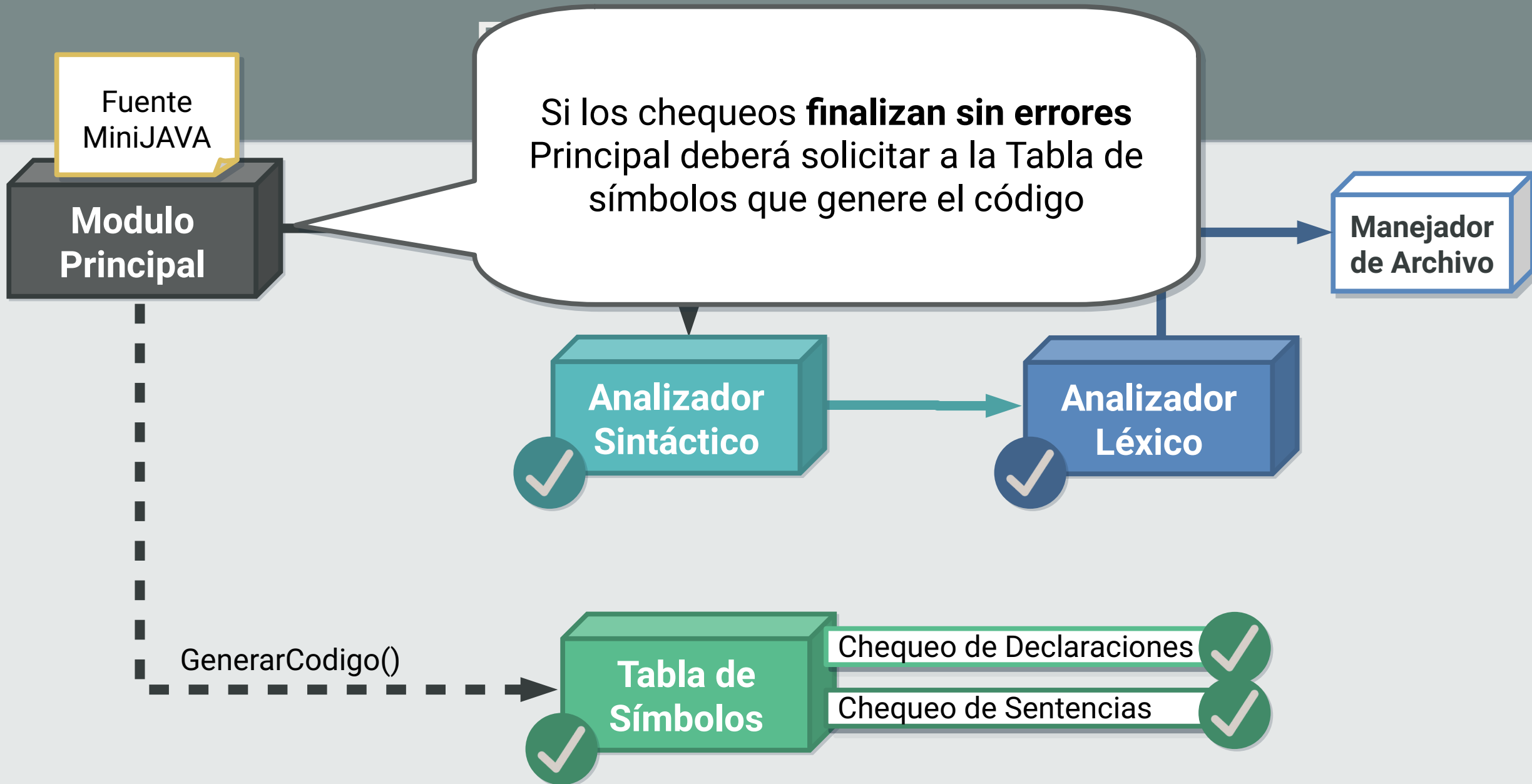


Esquema General

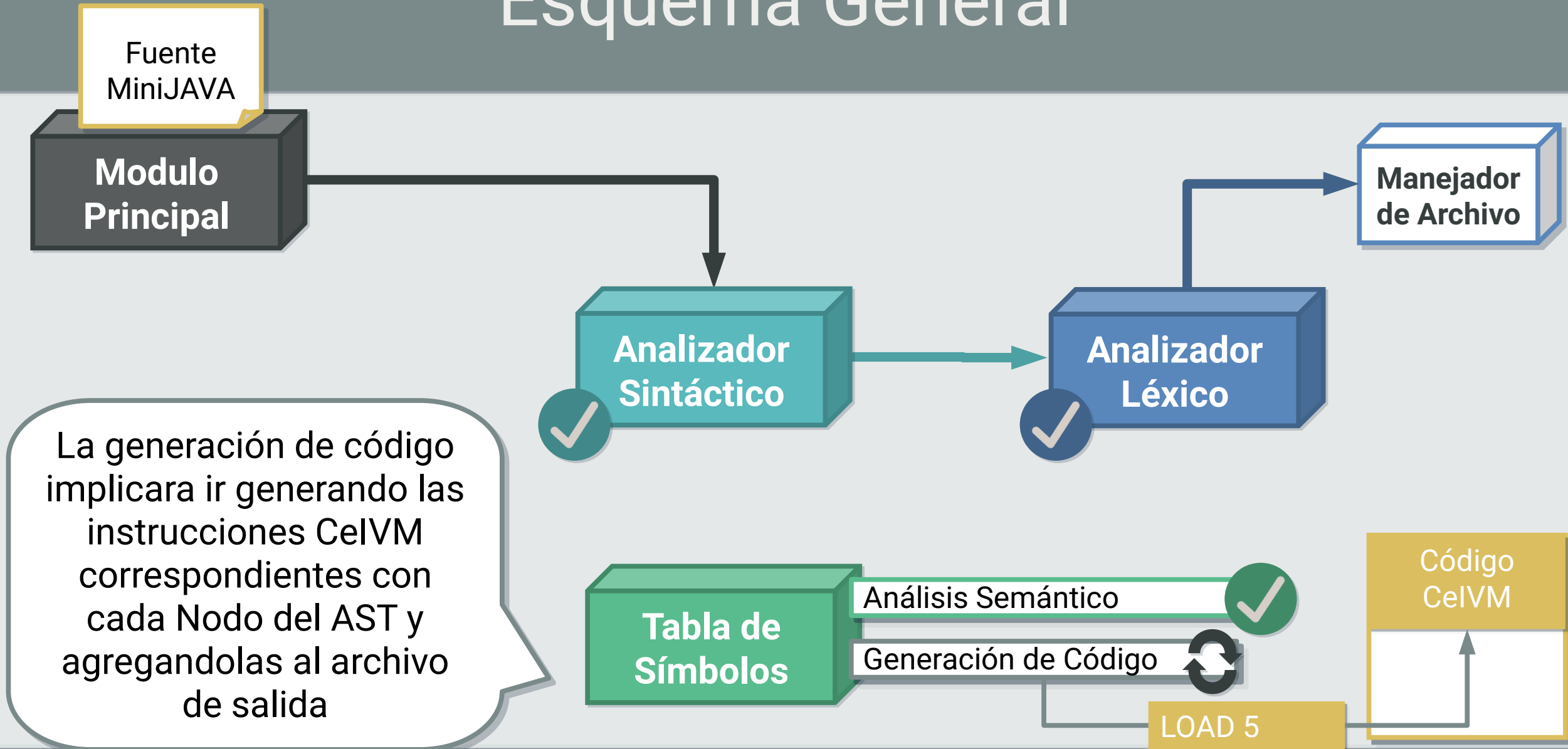


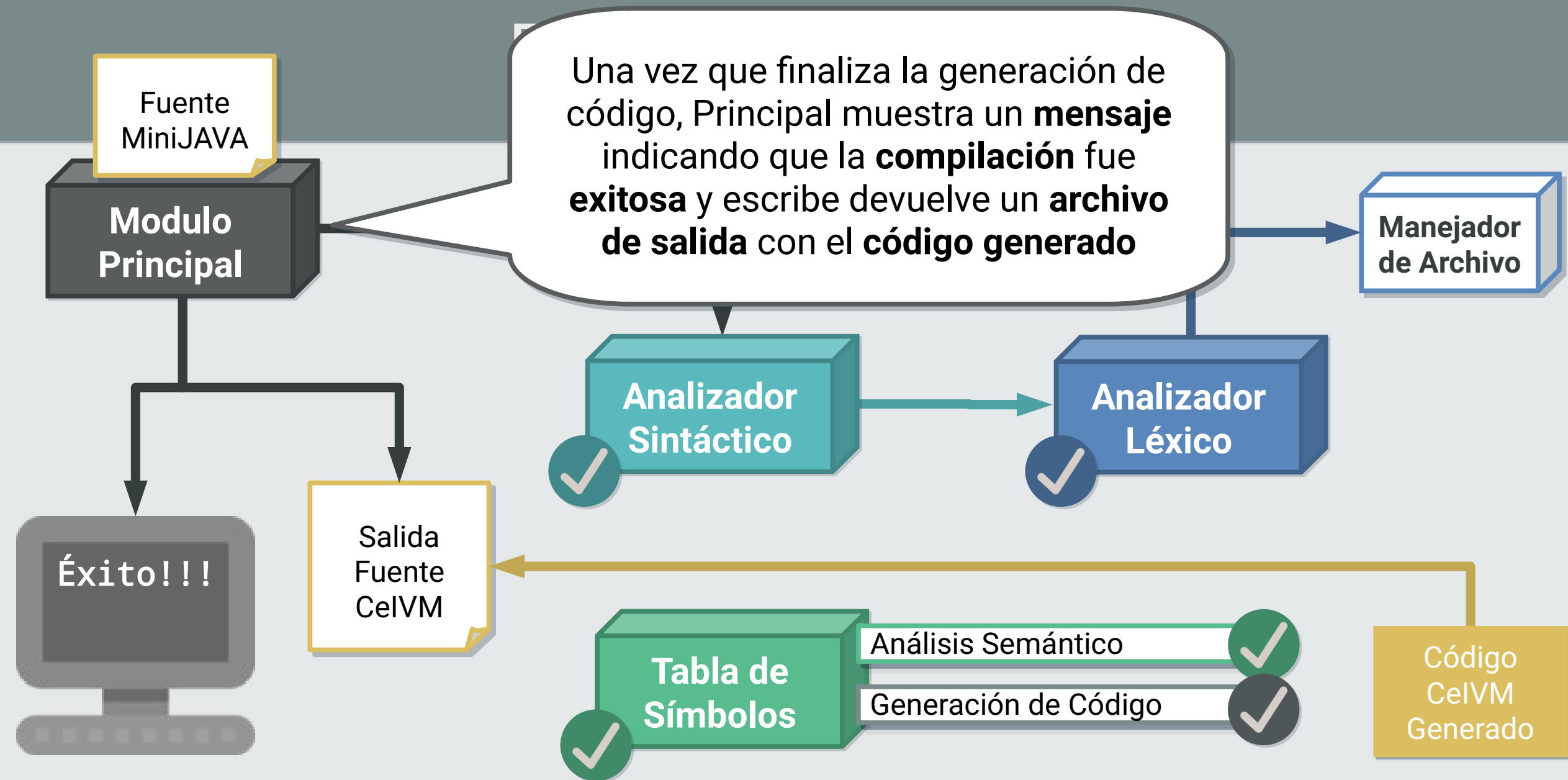


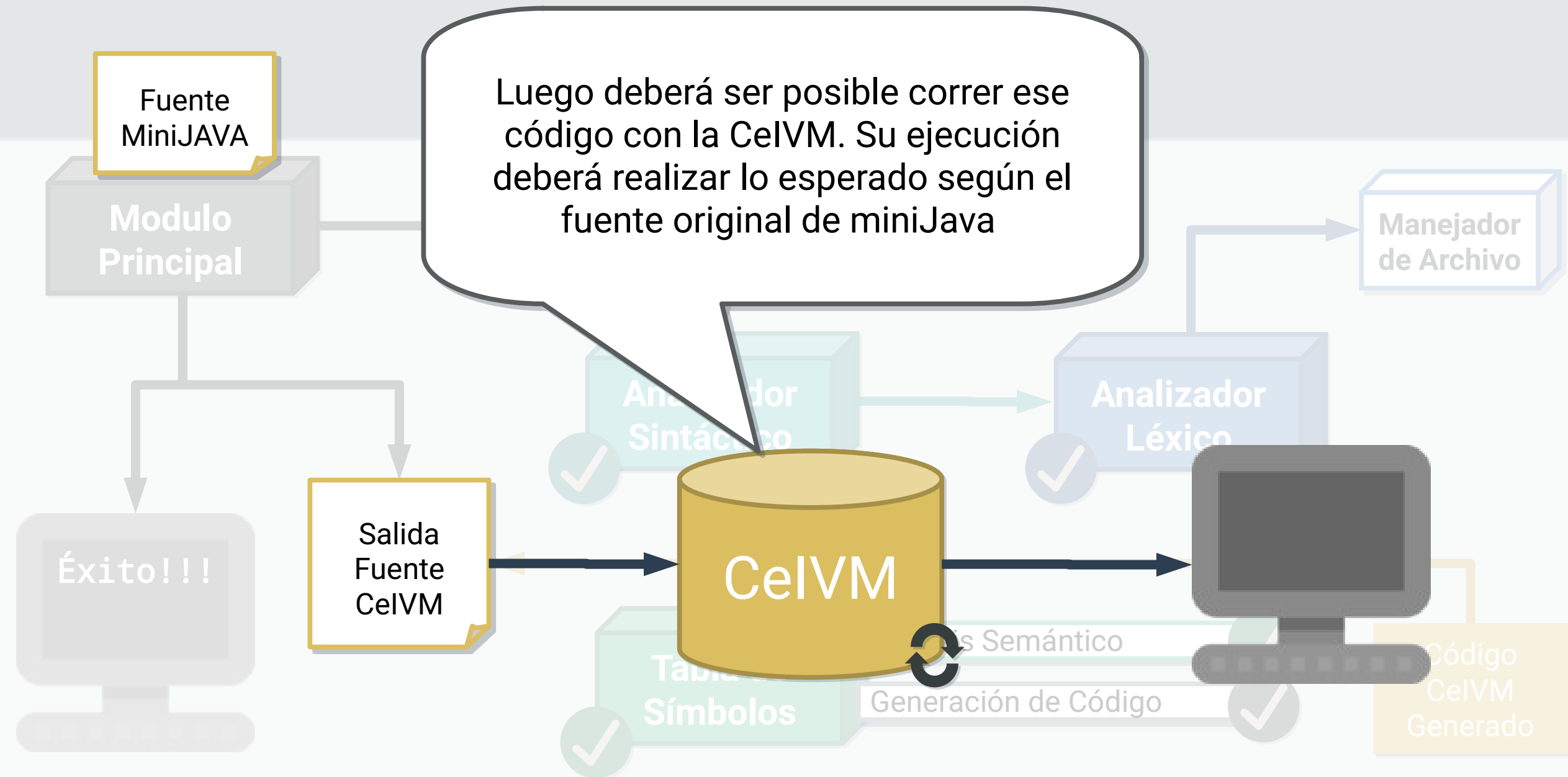


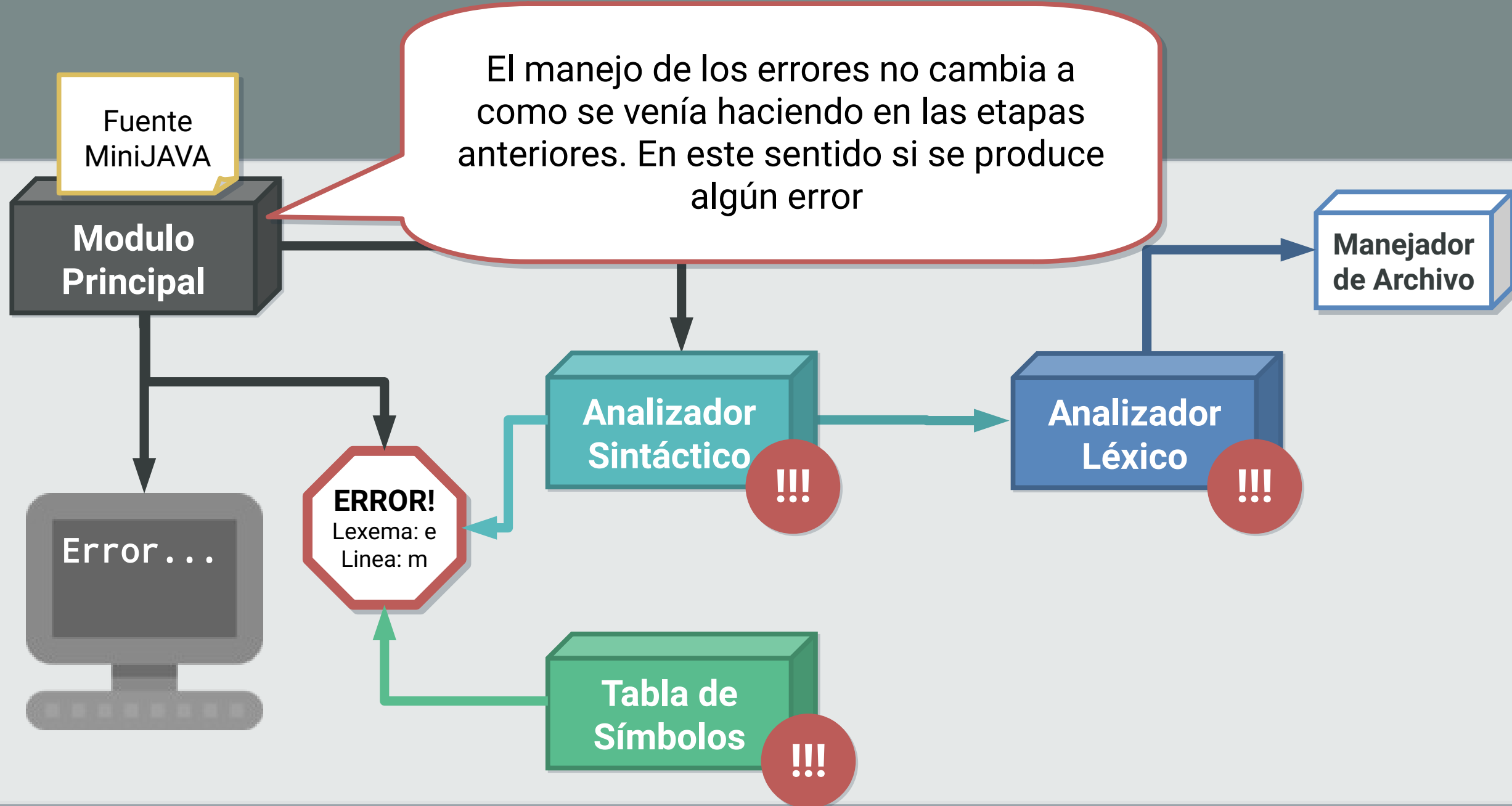


Esquema General









Convenciones y Consideraciones

Convenciones y Consideraciones

Fecha Limite de Entrega: 21/11/2025 a las 16hs

Forma de Entrega: En la actividad asociada a la entrega etapa 5 en Moodle, subiendo un zip con todo lo requerido

Convenciones y Consideraciones

La **interacción** de la aplicación entregada con el **usuario** deberá ser por **consola**



La entrega deberá consistir de: los archivos el **código fuente** (solo archivos **".java"**), un catálogo con los **casos de test** utilizados para la prueba del software y el **informe en formato pdf**



La **implementación** de la Generación de Código deberá corresponderse con los esquemas presentados en los Temas **5.2, 5.3.1, 5.3.2, 5.3.3 y 5.4**



Convenciones y Consideraciones

Mensaje de Análisis Exitoso

- Cuando no se detectan errores en el análisis el Modulo Principal debería mostrar un mensaje adecuado y luego el código de éxito: **[SinErrores]**
- Por ejemplo

Fuente MiniJava

```
class C1{  
}
```

El mensaje
debería ser

Compilacion Exitosa
[SinErrores]



Convenciones y Consideraciones

Invocación del Compilador

- Al ser compilado el software entregado poder invocarse de consola recibiendo como parámetros el archivo fuente MiniJava y el nombre del archivo de salida (**debe aceptar cualquier tipo de extensión!**)
- Por ejemplo: si al compilarlo lo llamamos Compilador, el fuente MiniJava se llama programa1.java y el archivo de salida para la CeIVM se llama salida.txt debería poder invocarse de la siguiente manera



```
java -jar Compilador.jar programa1.java salida.txt
```

Convenciones y Consideraciones



No cumplir con estas convenciones llevará a incurrir en fallos y/o a la desaprobación de la entrega/reentrega

Convenciones y Consideraciones



No cumplir con estas convenciones llevará a incurrir en fallos y/o a la desaprobación de la entrega/reentrega

Las convenciones son esenciales para que **nosotros podamos corregir** bien la entrega...
por eso somos tan estrictos!

Si no la podemos corregir
va a estar **desaprobada!!!**

Informe

Informe

- El informe deberá indicar **como compilar el código fuente** entregado y como correrlo una vez compilado.
- Debe presentarse el diagrama de clases de los ASTs y una breve descripción de que se genera en cada tipo de nodo.
- Deberá incluirse la **documentación** usual, propia de un **proyecto de software** (decisiones de diseño, clases utilizadas, etc).
- Debe indicarse que **logros** se intentan alcanzar en la Etapa

Logros Disponibles para la Etapa 5

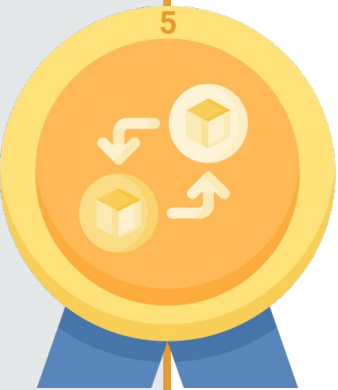
Logros Etapa 5

Requiere:



Coerción E5

El compilador genera el código adecuado para implementar la coerción permitida en la etapa anterior. En particular, la coerción de ints a floats chars a ints, chars a Strings y de ints a Strings (y viceversa). La coerción se aplica en los mismos contextos que en Java. Esto implica implementar también operaciones para la concatenación de Strings



Logros Etapa 5



toString E5

El compilador agrega el método void toString() a la clase Object. Este método se comporta de manera similar al de Java. De no ser redefinido, a diferencia de Java, solo muestra el nombre de la clase del objeto receptor por pantalla (en java muestra mas datos del objeto)

Logros Etapa 5



Entrega Anticipada Generación

(no valido para la reentrega)

La entrega debe realizarse 48hs antes de la fecha limite



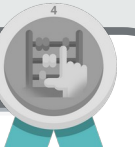
Imbatibilidad Generación

(no valido para la reentrega)

El software entregado pasa correctamente toda la batería de prueba utilizada por la cátedra

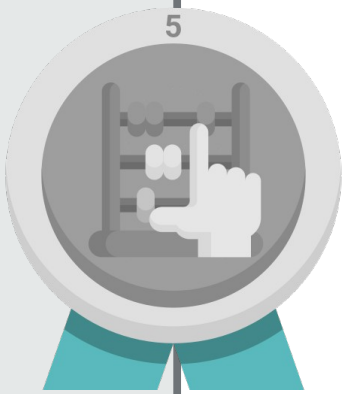
Logros Etapa 5

Requiere:



For Each - E5

El compilador genera el código adecuado para la sentencia for each. El comportamiento es similar al de Java, pero se vale de los métodos de la clase predefinida Miniterable. Al comenzar, invoca a start(), luego cicla mientras hasNext() sea verdadero. En cada iteración invoca a next(), y su resultado lo almacena en la variable de la izquierda de la sentencia for each.



Logros Etapa 5



Wrappers Primitivos E5

Requiere:



El compilador cuenta con las wrapper classes Integer, Character y Boolean. Estas son como en Java pero solo tienen el constructor que recibe el valor primitivo que wrapean y solo proveen el método dinámico para obtener el valor primitivo que están wrapeando. Su comportamiento y uso es el mismo que en Java.



Atributos Inicializados E5

Requiere:



El compilador Genera el código para inicializar los atributos cuando se invoca un constructor del objeto adecuado.

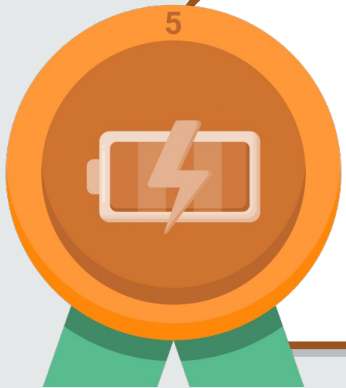
Logros Etapa 5



Super! - Ejecutado

El compilador genera el código adecuado para acceder a métodos de un ancestro usando **super** como en Java

Requiere:



Sobrecarga - Ejecutada

El compilador genera código para que se ejecuten adecuadamente los métodos y constructores sobrecargados.

Requiere:



Logros Etapa 5

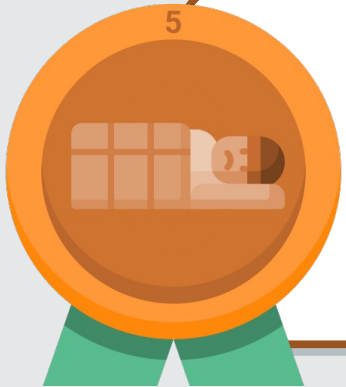


Requiere:



Generividad E5

El compilador ejecuta adecuadamente clases con generividad



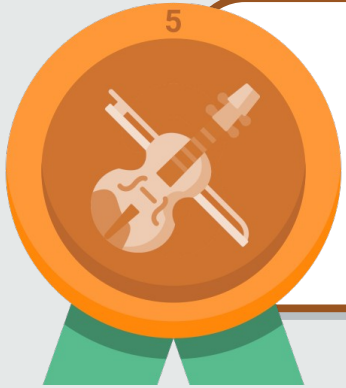
Requiere:



Atributos Tapados E5

Al igual que en java el compilador permite que una clase se declaren atributos con el mismo nombre que los definidos en sus superclases.

Logros Etapa 5



Variables Locales Clasicas E5

Requiere:



El compilador genera el código adecuado para reservar e inicializar (de ser necesario) variables locales declaradas de esta forma



For! Standard E5

Requiere:



El compilador genera código para ejecutar los for clásicos siguiendo la mecánica operacional de Java.

Logros Etapa 5



Ternario E5

El compilador implementa adecuadamente las expresiones con operadores ternarios

Requiere:



Logros de Etapas Anteriores

- Recordar que es posible también intentar alcanzar **logros** de la **etapa anterior**
 - **Es importante** que nos aclaren como desactivar el analizador semántico y/o la generación de código si persiguen logros del sintáctico, léxico, y/o semántico
- Aun así chequear que el logro de sea alcanzable en una etapa posterior ya que existen algunos que no lo permiten (como por ejemplo entrega anticipada o imbatibilidad)